Feature Stores, Data Pipelines & Offline/Online Serving

Scaler School of Technology | ML System Design and LLMOps | Week 2 — Session 1/2 | Post-Lecture Review & Practice

Key Takeaways

Data format choice affects performance

Parquet is not just a preference — it is 6× smaller and 2× faster to read than CSV on S3. Column-major layout means ML feature reads are efficient by design. Use row-major formats for writes and streaming; column-major for analytics and training.

Leakage is silent and deadly

Fitting your scaler on the full dataset before splitting is data leakage. So is using future information in a training feature. Both inflate offline metrics while producing models that fail the moment they see real data. The rule: split first, *then* compute all feature statistics on the training split only.

Feature stores solve the training-serving skew problem

Without a feature store, training code and serving code diverge. The same feature ends up computed differently in two places — a silent but catastrophic bug. A feature store defines features once, computes them consistently, and makes them available to both the offline (training) and online (serving) pipelines.

Streaming enables real-time ML

Batch features (total purchases last 30 days) can be precomputed nightly. Streaming features (transactions in the last 5 minutes) require a live stream processor like Flink. Knowing which of your features need streaming is a key design decision — streaming adds significant infrastructure complexity.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. Your team is running a Spark job to compute training features from a 500GB dataset stored on S3. A colleague suggests storing intermediate results as CSV for "human readability." Give two concrete reasons why Parquet would be a better choice here.

Q2. A data scientist trains a churn model and achieves 91% AUC on the validation set. When deployed, AUC drops to 67%. Post-mortem reveals the dataset was sorted by date and they used a random 80/20 split. What went wrong? What type of leakage is this?

Q3. Explain the difference between an OLTP database and an OLAP data warehouse. For a ride-sharing company, give one concrete example of a workload that belongs in each.

Q4. You are serving a recommendation model. The feature "number of videos watched in the last 1 hour" must reflect activity from the past 60 minutes in real time. Should this feature live in the offline store, the online store, or both? Justify your answer.

Design Exercises

Work through these independently or in pairs. No starter code — derive the approach from the lecture.

Exercise 1: Feature Pipeline for a Ride-Sharing ETA Model

Design a feature pipeline for a model that predicts a driver's estimated time of arrival (ETA). List at least 6 features. For each feature, classify it as either **batch** (can be precomputed daily or hourly) or **streaming** (must reflect near-real-time data). Sketch the data sources for each: which come from OLTP, which from a stream (Kafka), which from an external API. Identify which features require a join across multiple sources.

Exercise 2: Correct Train/Val/Test Split with a Timestamp Column

You have a dataset of 2 years of customer transactions with a `transaction_date` column. A colleague proposes a random 70/15/15 split.

Explain precisely why a random split would cause data leakage for any time-based feature (e.g., "user's purchases in the past 7 days"). Describe the correct approach: how would you split by time, and what date boundaries would you choose? What does your validation set measure that the training set cannot?

Challenge Problem

Design the Data Stack for a Fraud Detection System

You are the lead ML engineer at a fintech company. The fraud detection model must make a decision within 200 ms of every transaction. Design the full data stack:

(1) Data sources: What data do you ingest? From where (user events, bank APIs, device signals)? Batch or stream?

(2) Storage formats and systems: Where does raw data land? What format? Which OLTP and OLAP systems do you use?

(3) Feature engineering: List 5 features. For each: is it batch or streaming? What is the staleness budget (how old can it be)?

(4) Feature store design: What goes in the offline store vs. online store? How do you keep them in sync? What is your point-in-time strategy for training data generation?

(5) Serving: How does the model retrieve features at inference time? What is the latency breakdown across feature lookup, model inference, and response?

One paragraph per section. Justify every choice.

Further Reading

- **Chip Huyen — Designing Machine Learning Systems, Ch. 3 and Ch. 5** — O'Reilly, 2022 (primary source for today)
  Chapter 3 covers data engineering fundamentals: sources, formats, models, storage engines, data passing modes. Chapter 5 covers feature engineering in depth, including missing values, scaling, data leakage, and feature crossing.
- **Feast Documentation — feast.dev** — Open source feature store
  The quickstart takes 15 minutes and covers defining feature views, materializing to offline/online stores, and point-in-time retrieval. Highly recommended before the Week 2 Session 2 lab.
- **Martin Kleppmann — Designing Data-Intensive Applications, Ch. 3** — O'Reilly, 2017
  The definitive reference on storage engines, row vs. column-oriented formats, OLTP vs. OLAP, and data encoding. Chapters 3 and 4 are directly relevant.
- **Maxime Beauchemin — Functional Data Engineering** — medium.com, 2018
  Seminal post on idempotent pipelines and immutable data. Explains why point-in-time correctness matters and how to build pipelines that produce the same output every run.

Online Resources

- **Feast Docs** — feast.dev — open source feature store; quickstart in 15 min
- **Tecton Blog** — tecton.ai/blog — real-world feature store case studies
- **Hopsworks Blog** — hopsworks.ai — streaming features and MLOps
- **DuckDB** — duckdb.org — in-process OLAP on Parquet; great for local feature dev
- **Apache Kafka Docs** — kafka.apache.org — event streaming platform reference
- **Chip Huyen's Blog** — huyenchip.com — ML system design war stories

Answers

A1. Two reasons to prefer Parquet over CSV for a 500GB Spark job: (1) **Storage**: Parquet uses columnar compression that typically achieves 6× smaller files than CSV — 500GB becomes roughly 80GB, cutting S3 storage cost and network I/O. (2) **Speed**: Parquet supports predicate pushdown and column pruning — Spark reads only the columns needed, whereas CSV forces a full scan of every row. For feature computation queries that access 5–10 columns out of 100, this is a 10–20× speedup.

A2. The validation set contains rows from dates *before* the training set (because the random split mixed time periods). Any time-based feature computed during training (e.g., "purchases in past 7 days") uses future data that the model would not have at prediction time. This is **temporal data leakage**: future information leaked into the training window. The fix: sort by date, split at a cutoff. Training uses rows before cutoff, validation uses rows after.

A3. OLTP (e.g., PostgreSQL) is optimised for writes and point lookups: low latency, high concurrency, ACID transactions. Example: recording a ride booking in real time when a user confirms a trip. OLAP (e.g., BigQuery) is optimised for aggregate reads over large volumes. Example: computing the average fare per city over the past 30 days to feed a pricing model.

A4. Both. The offline store holds historical versions of this feature (computed from past logs) for point-in-time correct training data generation. The online store holds the *current* value (updated by a streaming job reading real-time watch events) so the serving system can look it up in sub-millisecond at inference time. Offline = training correctness; online = serving freshness.